DiagDesk

Design Document

HLD · LLD · Architecture & Data Model · Scale. Microservices on PostgreSQL · offline-first · India-sovereign · security & observability from day one.

C4 + sequence + ERD | db-per-service + RLS | sharding + partitioning | anti-kickback compliant

Engineering Design Reference

Confidential · 2026

OVERVIEW

Principles & non-functional targets

DiagDesk

Right-sized microservices

~12 DDD bounded contexts, hexagonal per service, database-per-service. No shared DB; events + sagas, not synchronous chains.

Multi-tenant by RLS

tenant_id + Postgres Row-Level Security; claims propagate gateway → service → DB. Large tenants isolatable.

Offline-first edge

Branch nodes keep the counter running through outages; conflict-aware, resumable sync.

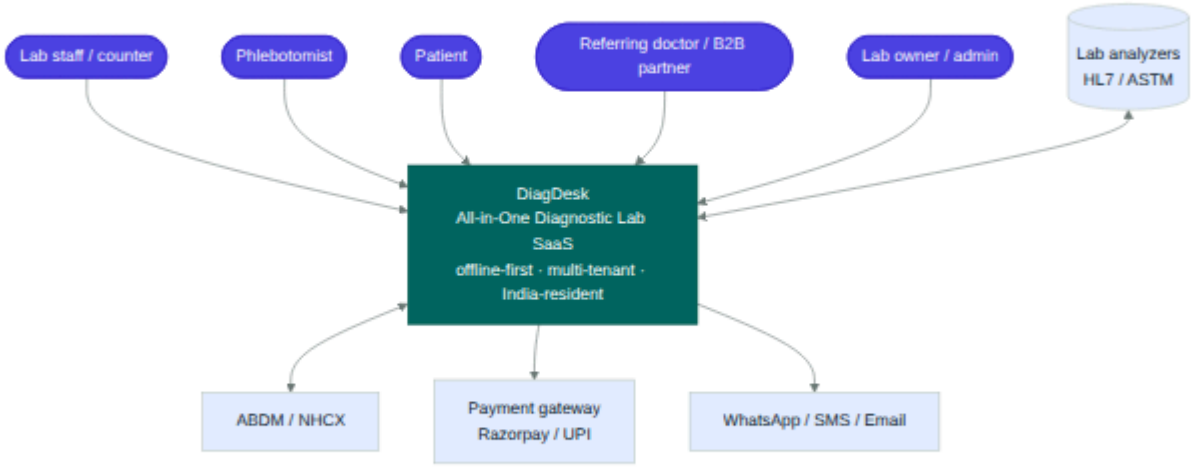
Scales horizontally

Sharding (tenant_id) × partitioning (time) × replicas/CQRS — to millions of records & high txn/min.

NFR targets: counter ops < 1s p95 (offline) · cloud OLTP write < 50ms p95 · MIS < 500ms p95 · DPDP + CERT-In residency · SLOs on the four golden journeys · **no referral-commission tooling** (ADR-007).

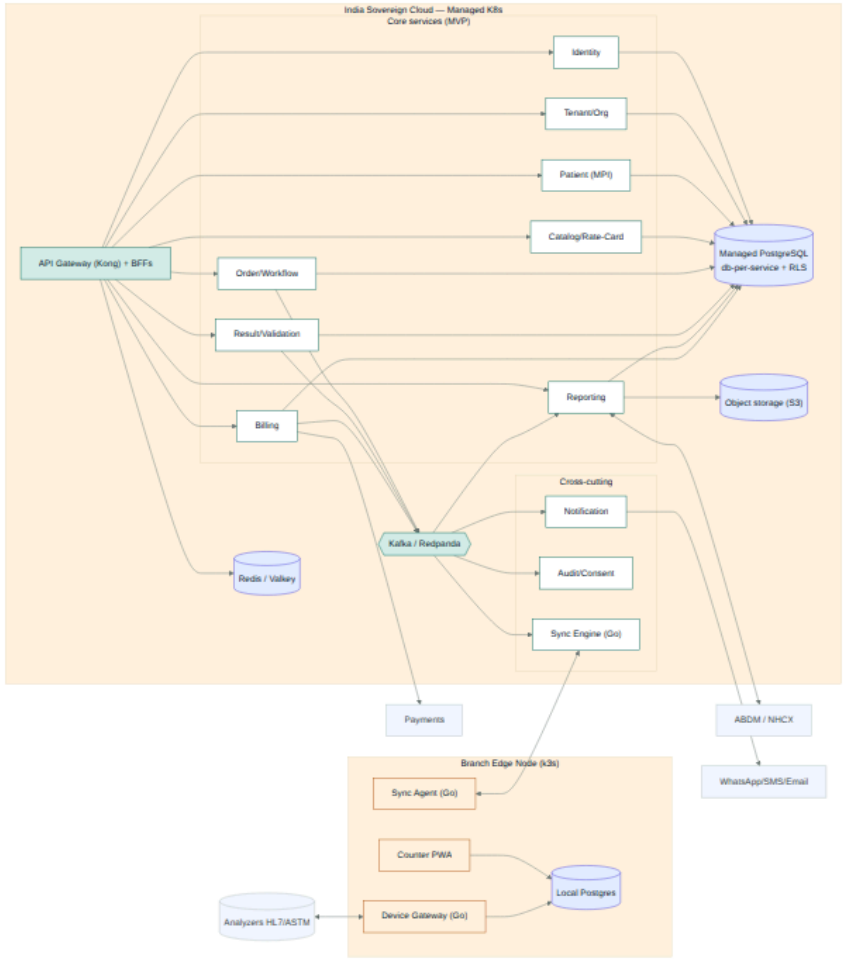
C4 · L1

System Context



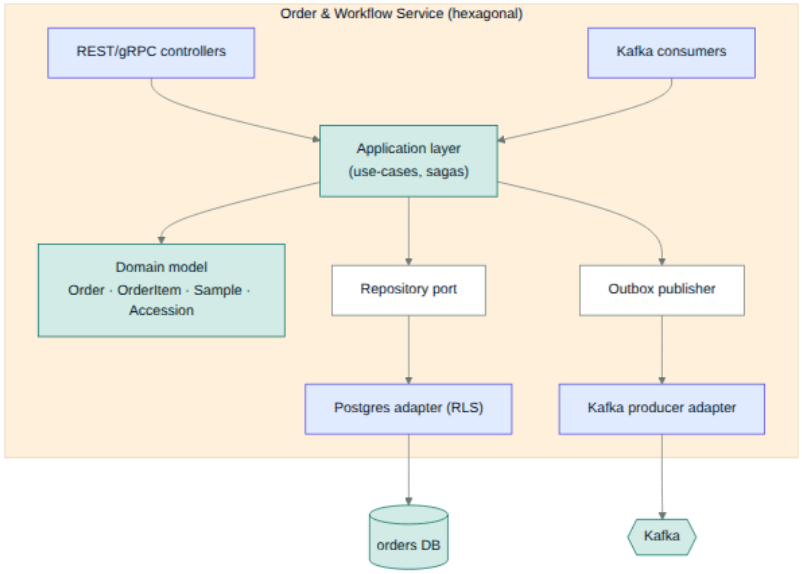
C4 · L2

Container / Architecture

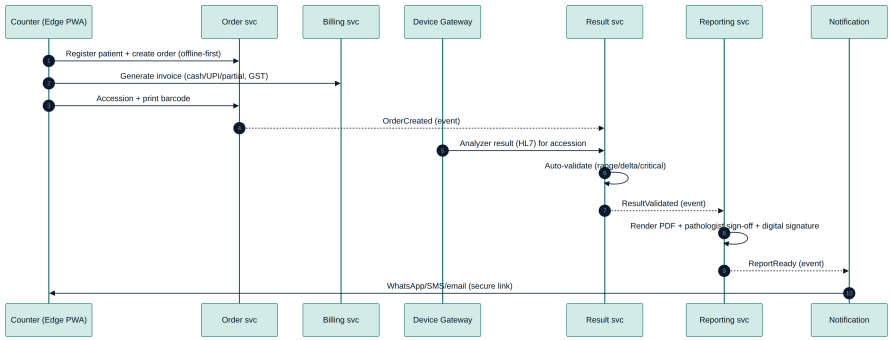


COMPONENT & FLOW

Service internals · Walk-in journey



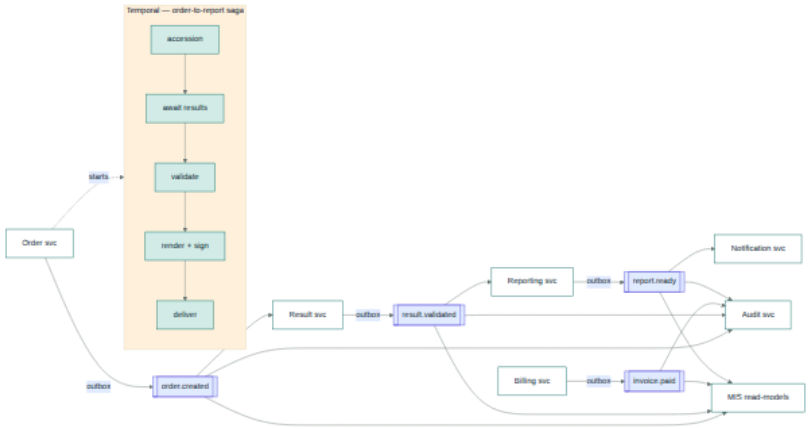
C4-L3 — Order & Workflow (hexagonal)



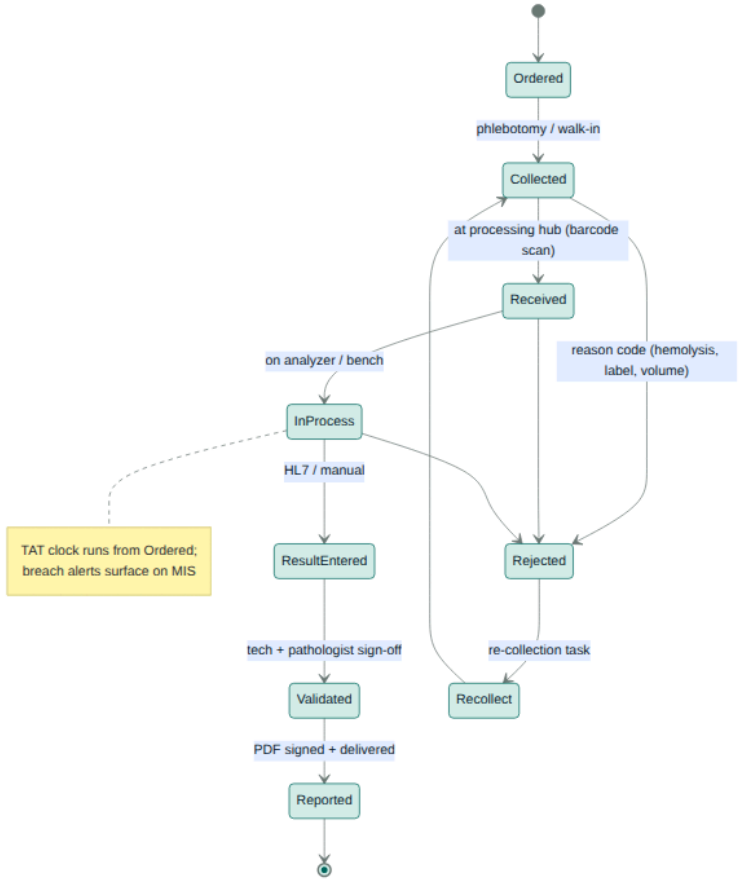
Sequence — register → bill → result → report

EVENTS & LIFECYCLE

Event/saga backbone · Sample state machine



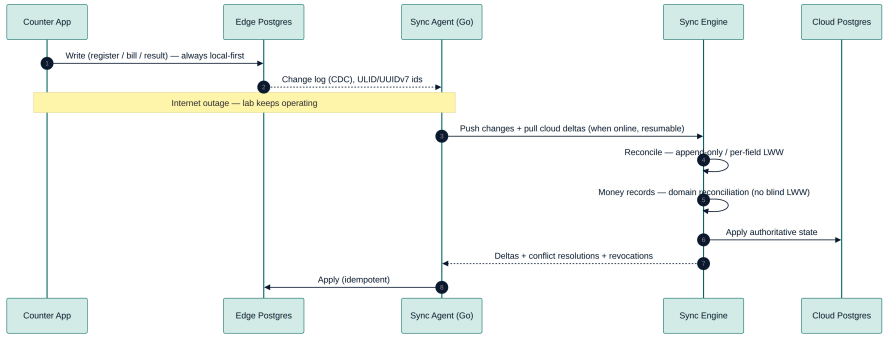
Kafka topics + Temporal order-to-report saga



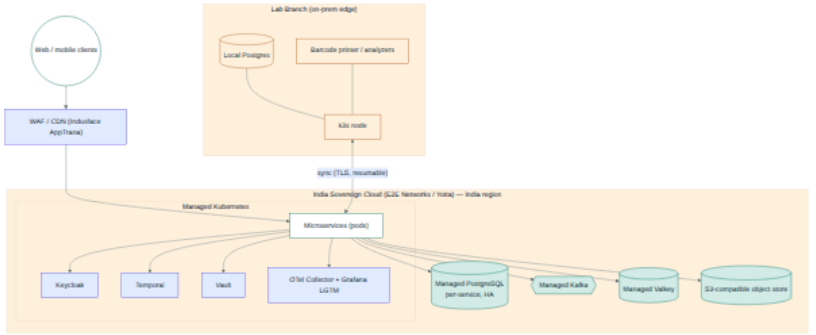
Sample lifecycle & TAT state machine

OFFLINE & DEPLOY

Offline-first sync · Deployment



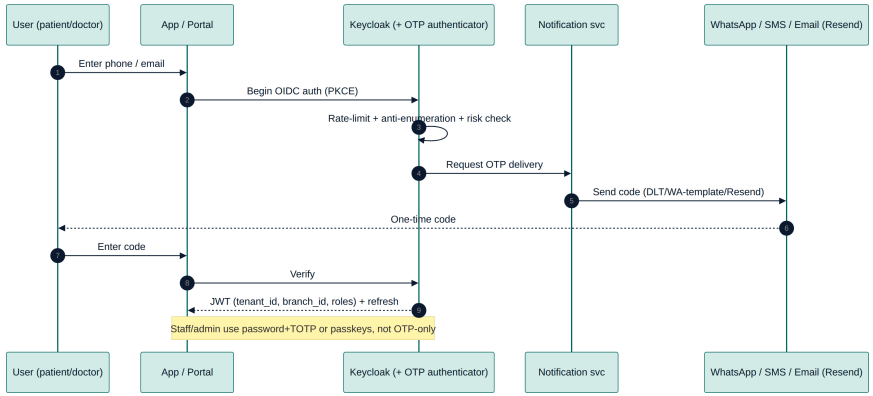
Edge ↔ cloud sync, money reconciliation



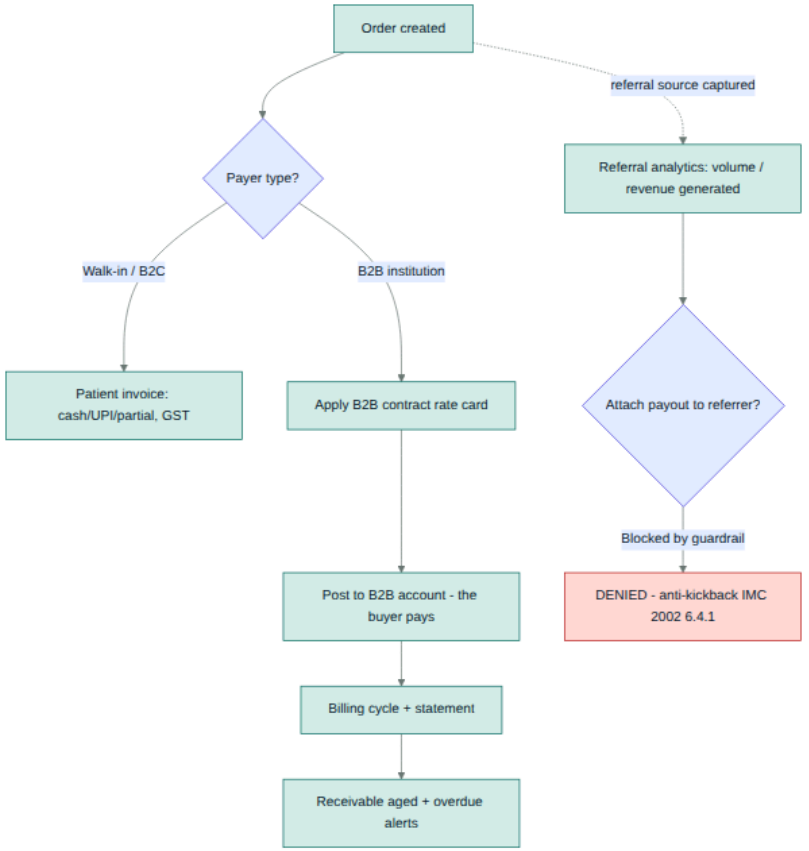
India sovereign cloud + branch edge (k3s)

AUTH & MONEY

OTP authentication · Compliant B2B billing



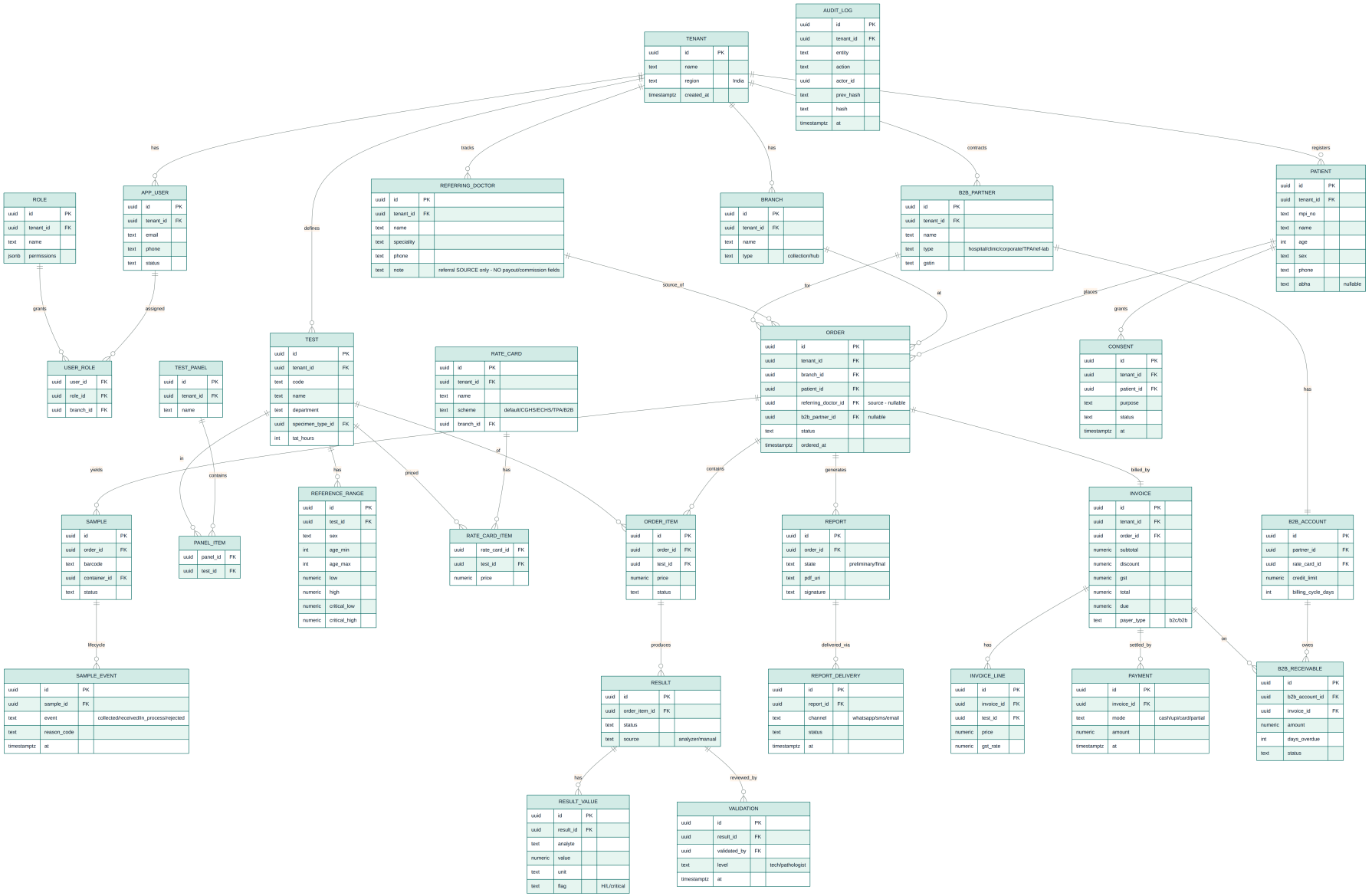
Keycloak OTP via WhatsApp/SMS/email



B2B billing — payout-to-referrer blocked (anti-kickback)

DATA MODEL

Entity-Relationship Diagram (MVP core)



SCHEMA

Data model — table groups

DiagDesk

Tenancy & platform: tenant, branch, app_user, role, user_role, consent, audit_log (hash-chained), notification, idempotency_key, outbox_event.

Master data: test, test_panel, panel_item, reference_range, specimen_type, container, rate_card, rate_card_item, report_template.

Parties: patient (MPI), referring_doctor *(referral source — no payout/commission fields)*, b2b_partner, b2b_account.

Orders & samples: order, order_item, sample, sample_event, accession.

Results & reports: result, result_value, validation, report, report_delivery.

Billing: invoice, invoice_line, payment, b2b_receivable.

Conventions: UUIDv7/ULID PKs · tenant_id + RLS on every tenant table · timestamptz · numeric(12,2) money · JSONB for FHIR/flexible · FK + tenant-leading composite indexes.

V1/V2 (described): qc_run, lj_point, inventory_item, stock_ledger, appointment, home_collection_visit, phlebotomist, abha_link, fhir_resource, nhcx_claim, study, pcpndt_form_f.

Compliance in the schema: there are **no commission/payout tables or columns anywhere**; referral sources are tracked for analytics only, and a billing guardrail blocks attaching a payout to a referrer (ADR-007).

SCALE

Millions of records & high txn/min

DiagDesk

Write throughput

- **Citus sharding** by tenant_id (co-located)
- **Monthly partitioning** of append tables
- **Kafka buffer** + batched inserts (COPY)
- Outbox + idempotent consumers
- **UUIDv7** = append-friendly indexes

Read & latency

- **Read replicas** for queries/reporting
- **CQRS read models** for dashboards
- **ClickHouse** for analytics at scale
- **Redis/Valkey** cache hot lookups
- OpenSearch for search

Ops & safety

- **PgBouncer** transaction pooling
- tenant-leading + partial + BRIN indexes
- HPA + Kafka-lag autoscaling
- Backpressure + DLQ + per-tenant limits
- Hot→warm→cold partition archival (DPDP)

Day-1 vs scale-out: MVP runs lean (one managed Postgres/service + PgBouncer + partitioning + cache + Kafka + a read replica). When metrics demand, add Citus shards, replicas and ClickHouse — no schema rewrite, because tenant_id, UUIDv7 PKs and time partition keys are baked in. See scalability-and-data-at-scale.md.